

Today 5:43 PM

 Perfect.

Next logical piece is:

How Next.js decides: "This route is static or dynamic?"

That connects everything we've covered:

```
fetch()  
cookies()  
headers()  
...etc
```

Show more ▾

Next.js Rendering Deep Dive — Lesson 6

How Next.js decides: Static or Dynamic?

YES. 🔥 This is exactly the right next piece because now we can connect **rendering + caching + SSG/SSR/ISR + CDN + Router Cache** into one mental model.

The key question is:

When Next.js builds or receives a request for `/products`, how does it decide whether it can pre-render that route or whether it must render it dynamically?

Let's build this from first principles.

1. First: What does "Static vs Dynamic" actually mean?

Suppose we have:

`/products`

There are two fundamentally different possibilities.

Static

Next.js can determine the HTML/RSC result **without knowing who the current visitor is or what their request contains**.

So it can essentially say:

"The result for `/products` can be generated ahead of time and reused."

Conceptually:

```
Build time
  ↓
Render /products
  ↓
HTML + RSC Payload
  ↓
Store in Full Route Cache
```



Then:

```
User A ┌─┐
User B ┌─┐ ──> cached result
User C ┌─┐
```



This is the world of:

- SSG
- ISR
- Full Route Cache
- potentially CDN caching

🔴 Dynamic

Next.js determines:

"I cannot know the correct response until I receive an actual request."

For example:

```
const cookieStore = await cookies()
```

The result could differ for every user.

Therefore:



```
graph TD; Browser --> Request; Request --> Nextjs[Next.js server]; Nextjs --> RenderNOW[Render NOW]; RenderNOW --> HTML[HTML + RSC Payload];
```

Browser
↓
Request
↓
Next.js server
↓
Render NOW
↓
HTML + RSC Payload

This is the world of:

- SSR
- request-time rendering
- generally no Full Route Cache for that dynamic response

2. The BIG distinction

Don't think:

Static = HTML file
Dynamic = no HTML

That's too simplistic.

Instead think:

Static



```
graph TD; Q1[Can Next.js safely reuse the rendered result?]; Q1 --> YES[YES]; YES --> R1[render ahead of time]; R1 --> C1[cache the result];
```

Can Next.js safely reuse the rendered result?
↓
YES
↓
render ahead of time
↓
cache the result

Dynamic



```
graph TD; Q2[Can Next.js safely reuse the rendered result?]; Q2 --> NO[NO]; NO --> R2[render per request];
```

Can Next.js safely reuse the rendered result?
↓
NO
↓
render per request

That's the fundamental decision.

3. So what signals does Next.js look at?

These are the important ones you listed:

```
fetch()  
cookies()  
headers()  
searchParams  
no-store  
revalidate  
generateStaticParams()
```



But they don't all mean exactly the same thing.

We need to divide them into **two categories**.

4. Category A — Request-dependent APIs

These are the big ones:

```
cookies()  
headers()
```

They tell Next.js:

"The output may depend on the incoming HTTP request."

For example:

```
export default async function Page() {  
  const cookieStore = await cookies()  
  
  const theme = cookieStore.get("theme")?.value  
  
  return <div>{theme}</div>  
}
```

Imagine:

```
User A  
Cookie: theme=dark
```



```
User B  
Cookie: theme=light
```

The page could produce:

```
User A → Dark  
User B → Light
```



So Next.js cannot safely generate one universal page ahead of time.

Therefore:

```
cookies()  
  ↓  
request-dependent data  
  ↓  
dynamic rendering
```



Same idea with:

```
headers()
```

because request headers can differ:

```
Authorization  
User-Agent  
Accept-Language  
etc.
```



5. What about `searchParams` ?

This one needs a little nuance.

Suppose:

```
/products?category=shoes
```



and:

```
/products?category=shirts
```



The rendered result might be different.

So query parameters can influence the rendered output.

In a page:

```
export default function Page({
  searchParams
}) {
  ...
}
```

you're saying:

"The page may use request-specific URL state."

But don't blindly memorize:

`searchParams = always dynamic`

Modern Next.js has more nuanced behavior depending on how the value is used and the caching/rendering configuration.

For interview purposes, remember:

Request-specific values such as cookies, headers, and URL search parameters are important signals that can make rendering request-dependent.

We'll come back to the exact `searchParams` behavior when we get to **Cache Components / newer Next.js rendering behavior**, because this is one area where Next.js versions have evolved.

6. Now the interesting one: `fetch()`

This is where people get confused. 🤔

They think:

```
fetch("/api/products")
```

means:

"Dynamic!"

NO.

`fetch()` itself does **not** automatically mean dynamic rendering.

The important question is:

How is this fetch configured/cached?

For example:

```
fetch("https://api.example.com/products", {  
  next: {  
    revalidate: 3600  
  }  
})
```

means:

"This data can be reused for up to an hour."

So Next.js can potentially render the route statically and cache the result.

Conceptually:

```
/products  
  ↓  
fetch products  
  ↓  
data can be cached  
  ↓  
route can be statically rendered  
  ↓  
Full Route Cache
```



7. `revalidate` is extremely important

Suppose:

```
fetch("https://api.example.com/products", {  
  next: {  
    revalidate: 3600  
  }  
})
```

```
}  
})
```

You are saying:

"The fetched data can remain fresh for 3600 seconds."

That's **ISR-style behavior**.

Conceptually:

```
Generate page  
  ↓  
Cache result  
  ↓  
1 hour  
  ↓  
stale  
  ↓  
revalidation  
  ↓  
new result
```



So:

```
revalidate  
  ↓  
cacheable data  
  ↓  
potentially static route  
  ↓  
Full Route Cache
```



8. Then what does `no-store` do?

Now:

```
fetch("https://api.example.com/products", {  
  cache: "no-store"  
})
```

means:

"Don't use the Data Cache for this fetch; get fresh data at request time."

Conceptually:

```
Browser request
  ↓
Next.js
  ↓
fetch(..., no-store)
  ↓
API
  ↓
fresh data
  ↓
render
```



This pushes the route toward **dynamic rendering**, because the result depends on request-time data.

So the simplified mental model is:

```
cacheable fetch
  ↓
static may be possible

no-store fetch
  ↓
dynamic rendering
```



9. Here's where `revalidate` and `no-store` differ

Think of them like this:

`no-store`

"Always get fresh data."

```
Request 1 → API
Request 2 → API
Request 3 → API
```



`revalidate: 60`

"Reuse the data for a while."

Request 1 → API



cache

Request 2 → cache

Request 3 → cache

...

after 60 sec



revalidate



That's the **Data Cache** layer.

10. But there's another important distinction

This is where our previous lessons become useful.

We have:

Data Cache



rendering



Full Route Cache



These are **not the same cache**.

For example:

```
const products = await fetch(..., {  
  next: { revalidate: 3600 }  
})
```

The fetched data may be cached in:

Data Cache



Then Next.js can use that data to produce:

HTML

+

RSC Payload

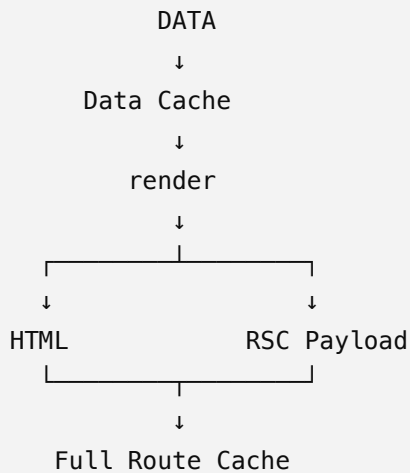


and cache the rendered route in:

Full Route Cache



So:



🔥 This distinction is absolutely critical.

11. What about `generateStaticParams()` ?

Now we get to dynamic routes.

Suppose:

`/products/[id]`

You have:

```
/products/1
/products/2
/products/3
...
```



How can Next.js know which pages it should statically generate?

You can provide:

```
export async function generateStaticParams() {
  return [
    { id: "1" },
    { id: "2" },
    { id: "3" }
  ]
}
```

```
]
}
```

Now Next.js knows:

```
/products/1
/products/2
/products/3
```



can be generated ahead of time.

Conceptually:

```
generateStaticParams()
  ↓
  known params
  ↓
  pre-render routes
  ↓
  Full Route Cache
```



This is particularly associated with **static generation of dynamic segments**.

12. Example: Blog

Imagine:

```
/blog/[slug]
```



You have:

```
/blog/nextjs
/blog/react
/blog/typescript
```



And:

```
export async function generateStaticParams() {
  const posts = await getPosts()

  return posts.map(post => ({
    slug: post.slug
```

```
    })))  
  }
```

Next.js can know:

```
nextjs  
react  
typescript
```



ahead of time.

So it can pre-render:

```
/blog/nextjs  
/blog/react  
/blog/typescript
```



🔥 This is classic static generation of dynamic routes.

13. Now let's put everything together

Imagine this page:

```
export default async function ProductsPage() {  
  const products = await fetch(  
    "https://api.example.com/products",  
    {  
      next: {  
        revalidate: 3600  
      }  
    }  
  )  
  
  return <Products products={products} />  
}
```

Ask Next.js:

Question 1

Does this depend on:

```
cookies?
```



No.

Question 2

Does it depend on:

headers?



No.

Question 3

Does it require:

request-time data?



No.

Question 4

Is the fetch cacheable?

Yes.

revalidate: 3600



Therefore:

Static rendering possible

↓

Generate result

↓

Full Route Cache

↓

CDN may cache HTTP response



That's essentially ISR behavior.

14. Now change ONE line

```
export default async function ProductsPage() {  
  const products = await fetch(  
    "https://api.example.com/products",  
    {  
      cache: "no-store"  
    }  
  )  
}
```

```
    return <Products products={products} />
  }
```

Now:

```
no-store
  ↓
request-time fetch
  ↓
route becomes dynamic
  ↓
render when request arrives
```



So:

```
Browser
  ↓
Next.js
  ↓
fetch fresh products
  ↓
render
  ↓
HTML + RSC Payload
  ↓
Browser
```



No reusable Full Route Cache entry for that dynamic response.

15. Now add `cookies()`

```
export default async function ProductsPage() {

  const cookieStore = await cookies()

  const userId = cookieStore.get("userId")

  const products = await fetch(...)

  return ...
}
```

Even if the fetch itself is cacheable, you've introduced:

```
cookies()  
↓  
request-specific information
```

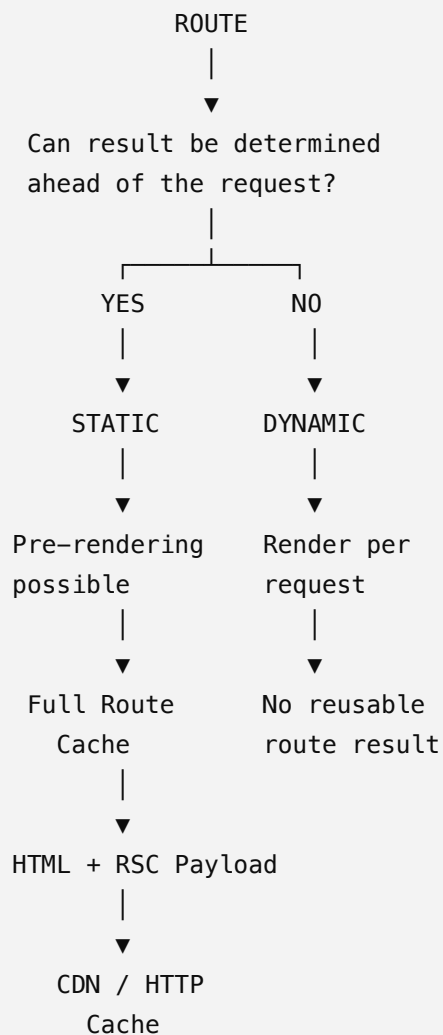


So the rendered output can depend on the visitor.

That's why this route is considered dynamic/request-dependent under the traditional App Router model.

16. The decision tree 🧠

Here's the mental model I want you to remember:



17. Where SSG, ISR and SSR fit

Now the terminology becomes MUCH easier.

SSG

Static generation where the result is generated ahead of time.

```
Build
↓
Render
↓
HTML + RSC
↓
Full Route Cache
```



ISR

Still fundamentally static/cached rendering, but the data/result can be regenerated periodically.

```
Generate
↓
Cache
↓
serve cached result
↓
revalidate
↓
regenerate
↓
new cached result
```



SSR

Request-time rendering.

```
Request
↓
Server
↓
fetch/request-specific data
↓
render
↓
response
```



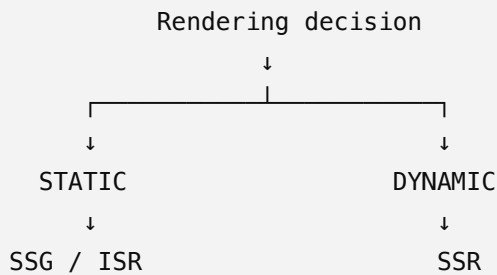
So don't think:

SSG / ISR / SSR



are completely separate magical mechanisms.

Think:



That's the cleaner mental model.

18. And now CDN enters the picture

This connects directly to your earlier question.

Suppose Next.js statically renders:

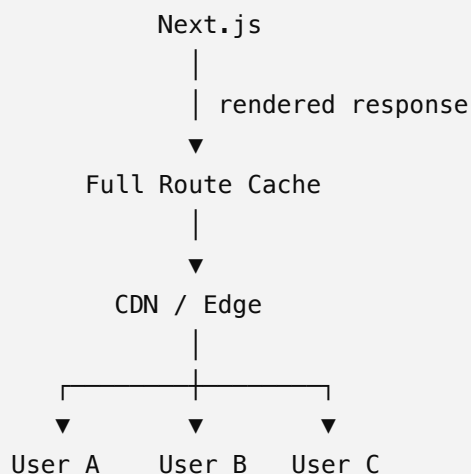
/products

and the result becomes:

Full Route Cache

The response can then potentially be served through a CDN/edge cache.

So conceptually:



But remember:

Next.js Full Route Cache and CDN cache are different layers.

This is exactly what we were discussing before.

19. And Router Cache?

Finally:

Server-side world

Data Cache



render



Full Route Cache



CDN / HTTP cache



Browser

Client-side world

Browser

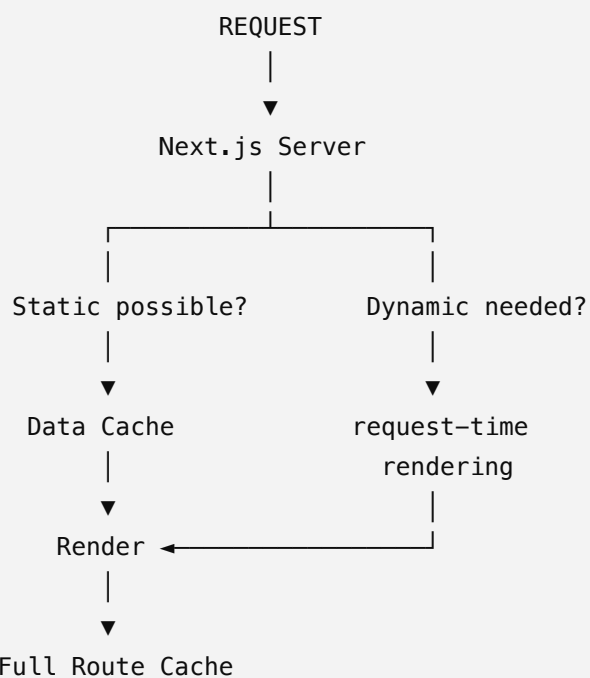


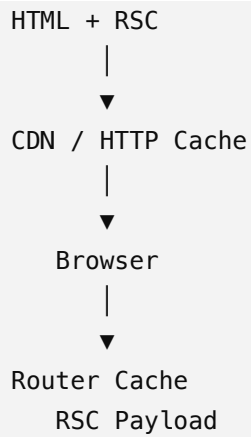
Router Cache



RSC Payload

So the complete picture becomes:





🔥 **THIS** is the big picture we've been building toward.

20. One important modern-Next.js caveat ⚠️

There's an important wrinkle in **newer Next.js versions**: the older mental model of "using a dynamic API automatically makes the whole route dynamic" is no longer the whole story with the newer **Cache Components / PPR-style architecture**.

So for your interview preparation, I recommend we learn this in two layers:

Layer 1 — Traditional App Router mental model

```

cookies()
headers()
no-store
request-dependent data
    ↓
Dynamic
  
```



and:

```

cacheable fetch
revalidate
generateStaticParams
    ↓
Static / ISR
  
```



Layer 2 — Modern Next.js

We'll then learn:

```

Cache Components
PPR
"use cache"
  
```



Suspense boundaries
request-time APIs inside partially prerendered routes

That prevents you from learning an outdated rule and later having to unlearn it.

Your core mental model

If you remember **one thing** from this lesson, make it this:

Next.js is essentially asking: "Can I safely determine and reuse this route's result without waiting for a particular request?"

If yes:

```
STATIC
↓
SSG / ISR
↓
Full Route Cache
↓
possibly CDN
```



If no:

```
DYNAMIC
↓
SSR / request-time rendering
↓
render when request arrives
```



And the things that influence that decision include:

```
cookies()
headers()
searchParams
fetch caching
no-store
revalidate
generateStaticParams()
```

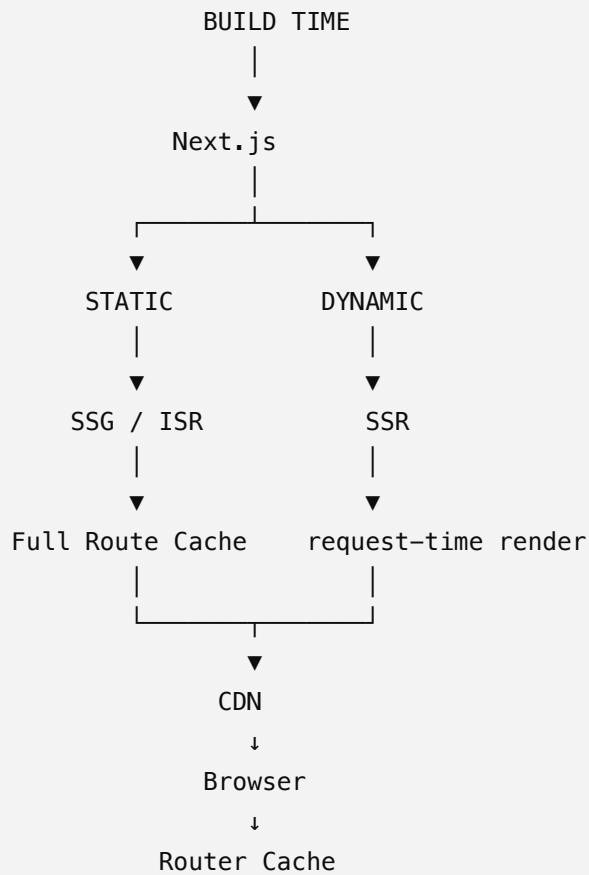


Next lesson

Now that we've answered "How does Next.js decide static vs dynamic?", the perfect next step is:

Lesson 7 — Static vs Dynamic Rendering: Under the Hood

We'll take the exact same `/products` page and trace:



And we'll also finally make **SSG vs SSR vs ISR** crystal clear with actual request timelines.



Today 8:19 PM

also lets learn both layer 1 and layer 2 mental modela

Absolutely. 👍 And I actually think **this is the better way to learn Next.js deeply.**

We'll learn **both mental models**, but in the right order so you don't end up mixing the older App Router model with the newer architecture.

Our plan 🧠

Layer 1 — Traditional App Router mental model

We'll master:

```

fetch()
cookies()
headers()
searchParams
cache: "no-store"
revalidate
generateStaticParams()

```

```

↓
Static vs Dynamic
↓
SSG / SSR / ISR
↓
Data Cache
↓
Full Route Cache
↓
CDN / HTTP Cache
↓
Router Cache

```

You'll understand **exactly why a route becomes static/dynamic**, when it renders, and what gets cached.

Layer 2 — Modern Next.js mental model

Then we'll learn the newer architecture:

Cache Components

```

|
├── "use cache"
|
├── Suspense
|
├── PPR / Partial Prerendering
|
├── request-time APIs
|
└── dynamic content
    ↓
  Static shell + dynamic parts

```

And importantly, we'll understand **why this architecture changes some of the older "dynamic API = entire route is dynamic" rules**.

The final mental model we want

By the end, you should be able to look at something like:

```
export default async function ProductPage({
  searchParams
}) {
  const user = await getUser()
  const products = await getProducts()

  return (
    <>
      <Header />
      <ProductList products={products} />
      <Recommendations user={user} />
    </>
  )
}
```

and reason through:

- What runs at build time?
- What runs at request time?
- What is cached?
- Where is it cached?
- Can the HTML be reused?
- Can only part of the page be dynamic?
- What does the browser receive?
- What does the Router Cache hold?
- What can the CDN cache?

That's the level we're aiming for. 🔥

And I won't just give you rules to memorize—we'll repeatedly trace **Browser → CDN → Next.js → Data Cache → rendering → Full Route Cache → RSC → Router Cache** so the whole thing becomes one connected system.

So yes: **Layer 1** first, then **Layer 2**, and finally we'll merge them into one modern **Next.js** rendering mental model.



Do you like this personality?



ChatGPT can make mistakes. Check important info.

